The Notion-Based Reasoning System

Compositional Execution via Dependency-Gated Recurrence

Aryan Gupta
aryan.cs.app@gmail.com

May 19, 2026

Abstract

The Notion-Based Reasoning System (NBRS) performs reasoning by traversing a typed directed acyclic graph of operations and resolving each operation node with a shared, capacity-limited neural processor invoked only when its inputs are available. This document develops the formal theory of the architecture. We formalise the executor as a deterministic state machine over a typed DAG and establish its core operational properties: argument availability is preserved as an invariant, the resulting computation is deterministic, order-independent across topological schedules, terminates in a number of node invocations linear in graph size, and reduces to oracle evaluation under a primitive-accurate processor. We contrast its inductive bias with a weight-tied graph transformer carrying a causal ancestor mask and with a sequence transformer over a serialised DAG, and prove an expressivity containment between the three. The executor's per-step input is bounded by the maximum primitive arity independently of graph depth; combined with a stochastic processor of per-invocation error ε, this yields a capacity–depth decoupling: depth-d graphs are resolved correctly with probability at least $(1-\varepsilon)^d$ and per-step parameter count is constant in d. The processor sees only the local context of each node, which gives the architecture a clean local-context invariance principle and, under marginal-invariance of the local context across depths, a depth-independent expected per-node error. We extend the framework to autonomous graph construction as a Markov decision process with deterministic transitions and a polynomially-bounded action space, and we state the formal questions associated with extending the primitive library through subgraph abstraction.

1 Introduction

A neural network that simulates multi-step reasoning through autoregressive token prediction must perform two distinct functions simultaneously: deciding what sub-problems to solve, and solving them. The two functions share parameters, share context, and share representational bandwidth. As compositional depth grows, the constraints they place on each other tighten. This is the structural reason behind the observed degradation of token-level reasoning with depth: a fixed-capacity sequence model cannot allocate independent computational resources to each step of a multi-step computation.

The NBRS separates the two functions architecturally. Reasoning is represented as a typed directed acyclic graph of *notions*—operations on typed values—with edges denoting argument dependencies. Execution is dependency-gated: a node fires only when all of its arguments are resolved, and at that point a single, shared, capacity-limited neural module (the *node processor*) is invoked on the local context of the node. The same parameters are reused at every node, unrolled in time rather than in width. Compositional depth grows the number of node invocations; it does not consume neural parameters, attention positions, or representational bandwidth.

This document develops the theory. We define the typed DAG and its operational semantics, prove the executor's invariants, derive its information-theoretic properties, and characterise the

conditions under which depth-generalisation holds. We then formulate the natural extensions: autonomous graph construction by a learned policy, and extension of the primitive library through subgraph abstraction. Throughout, the goal is to produce statements that admit precise analysis and clean comparison with sequence-based and graph-based alternatives.

2 Formal Preliminaries

We work in a typed first-order setting. Values have types; operations have input and output types; reasoning is a typed computation organised as a DAG.

2.1 Types and values

Definition 2.1 (Type system)**.** A type system is a pair $(\mathcal{T}, \mathcal{V})$ where $\mathcal{T}$ is a finite, decidable set of types and $\mathcal{V} = \bigsqcup_{\tau \in \mathcal{T}} \mathcal{V}_\tau$ is a disjoint union of value domains. We write $v : \tau$ for $v \in \mathcal{V}_\tau$.

2.2 Primitive operations

Definition 2.2 (Primitive operation)**.** A primitive operation is a tuple

$$\tau \;=\; (\mathsf{name}_\tau,\ \mathsf{in}_\tau,\ \mathsf{out}_\tau,\ f_\tau),$$

where $\mathsf{in}_\tau \in \mathcal{T}^*$ is a (possibly empty) tuple of input types of length k_τ, $\mathsf{out}_\tau \in \mathcal{T}$ is the output type, and f_τ is a total function

$$f_\tau \;:\; \mathcal{V}_{\tau_1} \times \cdots \times \mathcal{V}_{\tau_{k_\tau}} \;\to\; \mathcal{V}_{\mathsf{out}_\tau}.$$

We refer to f_τ as the *semantic function* of τ; it defines the mathematical meaning of the primitive, independently of any neural realisation.

Definition 2.3 (Primitive library)**.** A primitive library is a finite, ordered set $\mathcal{L} = \{\tau_1, \ldots, \tau_K\}$ of primitives. The library is type-closed if for every $\tau \in \mathcal{L}$ and every $\sigma \in \mathsf{in}_\tau$ either σ is an input type of the overall task or some primitive in $\mathcal{L}$ has output type σ.

2.3 Notions and notion DAGs

Definition 2.4 (Notion)**.** A notion over a library $\mathcal{L}$ is a tuple

$$n \;=\; (\mathsf{id}_n,\ \tau_n,\ \mathsf{spec}_n,\ \mathsf{pa}_n),$$

where id_n is a unique identifier, $\tau_n \in \mathcal{L}$, spec_n encodes any auxiliary parameters of the primitive (constants, shift offsets, comparison directions), and $\mathsf{pa}_n = (p_1, \ldots, p_{k_{\tau_n}})$ is the ordered tuple of parent identifiers, one per input slot of τ_n. A notion with $\mathsf{pa}_n = ()$ is a leaf.

Definition 2.5 (Notion DAG)**.** A notion DAG over $\mathcal{L}$ is a pair $G = (V, r)$ where V is a finite set of notions, every parent identifier in $\bigcup_{n \in V} \mathsf{pa}_n$ refers to a notion in V, the induced parent relation is acyclic, and $r \in V$ is a distinguished output notion.

Definition 2.6 (Well-formedness)**.** A notion DAG $G = (V, r)$ is well-formed if (i) the parent relation is acyclic; (ii) every parent identifier in pa_n refers to a notion in V; (iii) types match along every edge: for each $n \in V$ and each $i \in \{1, \ldots, k_{\tau_n}\}$, the i-th parent p_i satisfies $\mathsf{out}_{\tau_{p_i}}$ equal to the i-th declared input type of τ_n; (iv) every non-leaf notion has exactly k_{τ_n} parents.

All notion DAGs in the remainder of this document are assumed well-formed.

Definition 2.7 (Compositional depth)**.** The depth of a notion n in a notion DAG G is

$$\mathsf{depth}(n) \;=\; \begin{cases} 0 & \text{if } \mathsf{pa}_n = (), \\ 1 + \max_{p \in \mathsf{pa}_n} \mathsf{depth}(p) & \text{otherwise.} \end{cases}$$

The depth of G is $\mathsf{depth}(G) := \mathsf{depth}(r)$.

A notion DAG generalises an abstract syntax tree by permitting value sharing: a single resolved value may feed multiple downstream operations. The remainder of this document handles the general DAG case.

2.4 Reasoning tasks and oracle execution

Definition 2.8 (Reasoning task)**.** A reasoning task over $\mathcal{L}$ is a pair (G, σ) where $G = (V, r)$ is a well-formed notion DAG and $\sigma : V_{\text{leaf}} \to \mathcal{V}$ is a type-respecting assignment of values to the leaves of G. The ground-truth answer of the task is the value at the root produced by recursive substitution of the primitive semantic functions.

Recursive substitution evaluates each non-leaf notion n as

$$\hat{v}_n \;=\; f_{\tau_n}(\hat{v}_{p_1}, \ldots, \hat{v}_{p_{k_{\tau_n}}}),$$

where $(p_1, \ldots, p_{k_{\tau_n}}) = \mathsf{pa}_n$. We record the order-independence of this evaluation for later use.

Lemma 2.9 (Order independence of oracle execution)**.** *Let $G = (V, r)$ be a well-formed notion DAG with leaf assignment σ and primitive semantic functions $\{f_\tau\}_{\tau \in \mathcal{L}}$. For any two topological orderings π_1, π_2 of V, the value $\hat{v}_n$ produced at each $n \in V$ by oracle evaluation is the same under π_1 and π_2.*

Proof. Induction on $\mathsf{depth}(n)$. For $\mathsf{depth}(n) = 0$ we have $\hat{v}_n = \sigma(n)$, independent of order. For $\mathsf{depth}(n) > 0$, $\hat{v}_n$ depends only on f_{τ_n} and the parent values, each of which is order-invariant by the inductive hypothesis. Determinism of f_{τ_n} yields the conclusion. □

3 The Graph-Structured Recurrent Executor

The Graph-Structured Recurrent Executor (GSRE) is a deterministic state machine over a notion DAG that delegates value resolution at each node to a shared neural processor. This section defines the executor formally and establishes its core operational properties.

3.1 Node states

Definition 3.1 (Node state)**.** During execution, every node $n \in V$ is in one of five states drawn from $\mathcal{S} = \{\textsc{Pending}, \textsc{Ready}, \textsc{Executing}, \textsc{Resolved}, \textsc{Failed}\}$, written $\mathsf{state}(n) \in \mathcal{S}$. Once $\mathsf{state}(n) = \textsc{Resolved}$, the associated value $v_n \in \mathcal{V}_{\mathsf{out}_{\tau_n}}$ is fixed for the remainder of execution.

The gating of $\textsc{Pending} \to \textsc{Ready}$ on parent state, depicted in Figure 1, is the architectural source of the executor's inductive bias: dependency order is not a learned soft attention pattern, it is a hard transition rule.

3.2 The traversal engine

The traversal engine drives the state machine. At each step it identifies all ready nodes, calls the shared processor on each, and updates state.

Algorithm 1 batches all currently-ready nodes per step. The batch composition is immaterial to correctness, as we will show.

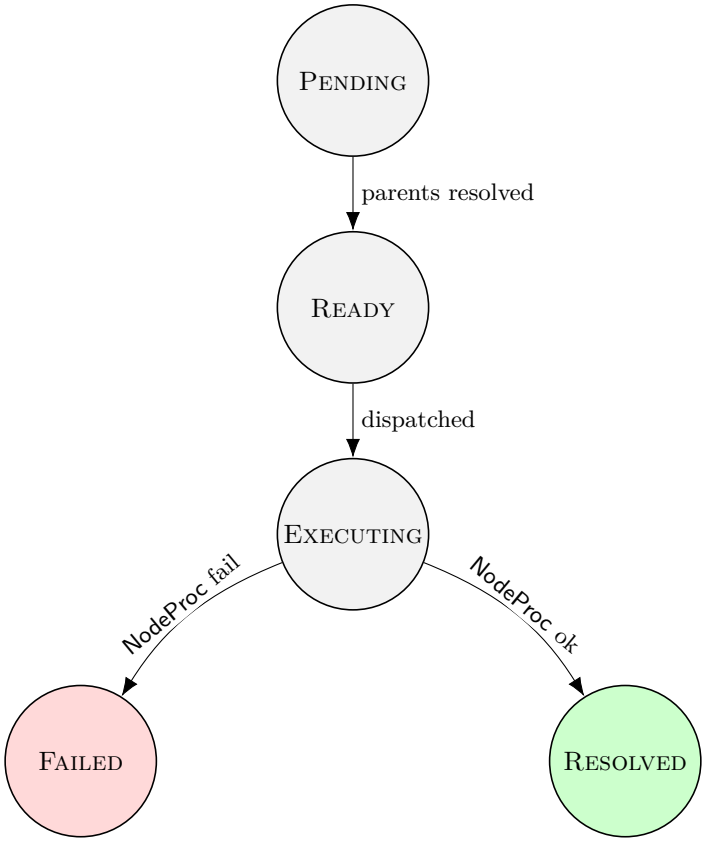


Figure 1: The node-state machine. PENDING → READY is gated by a hard predicate on parent state; READY → EXECUTING by dispatch; EXECUTING → RESOLVED or EXECUTING → FAILED by the outcome of the node-processor call.

Lemma 3.2 (Local input determinism)**.** *At line 14 of Algorithm 1, the input to* $\mathsf{NodeProc}_\theta$ *for any* $n \in \mathcal{B}$ *consists exclusively of* τ_n, spec_n, *and the resolved values of* pa_n. *It does not depend on the identity, type, depth, or value of any other node in the graph.*

Proof. By inspection of line 14. □

Lemma 3.3 (Argument-availability invariant)**.** *Throughout the execution of Algorithm 1, whenever* n *enters* EXECUTING *every* $p \in \mathsf{pa}_n$ *satisfies* $\mathsf{state}(p) =$ RESOLVED.

Proof. Transitions into EXECUTING occur only at line 13, reached only for $n \in \mathcal{B} \subseteq R$. The set R was constructed at line 4 to consist precisely of PENDING nodes whose parents are all RESOLVED. Once a node is RESOLVED its state is monotone, so the predicate continues to hold at line 13. □

3.3 Operational properties

Theorem 3.4 (Determinism)**.** *Let* $\mathsf{NodeProc}_\theta$ *be a deterministic function of its input. Then for any well-formed notion DAG* G *and any leaf assignment* σ, *the value assignment returned by Algorithm 1 is uniquely determined by* G, σ, *and* θ.

Proof. We show that v_n is uniquely determined for each n by induction on $\mathsf{depth}(n)$. The base case is line 2, which assigns $v_n = \sigma(n)$ to every leaf. For the inductive step, suppose every $p \in \mathsf{pa}_n$ has its value uniquely determined; Theorem 3.6 below ensures that n eventually enters R. At line 14, the input to $\mathsf{NodeProc}_\theta$ at n is $(\tau_n, \mathsf{spec}_n, (v_p)_{p \in \mathsf{pa}_n})$ (Lemma 3.2), each component

Algorithm 1 The Graph-Structured Recurrent Executor

Require: Well-formed notion DAG $G = (V, r)$ with leaf assignment σ; processor $\mathsf{NodeProc}_\theta$.
Ensure: A (possibly partial) value assignment $v : V \rightharpoonup \mathcal{V}$.

1: **for each** $n \in V$: $\mathsf{state}(n) \leftarrow$ PENDING
2: **for each** leaf $n \in V$: $v_n \leftarrow \sigma(n)$; $\mathsf{state}(n) \leftarrow$ RESOLVED
3: **loop**
4: $R \leftarrow \{n \in V : \mathsf{state}(n) = \text{PENDING}$ and all $p \in \mathsf{pa}_n$ have $\mathsf{state}(p) = \text{RESOLVED}\}$
5: **for each** $n \in R$: $\mathsf{state}(n) \leftarrow$ READY
6: **if** $R = \emptyset$ **then**
7: **if** $\mathsf{state}(r) =$ RESOLVED **then**
8: **return** v
9: **else if** $\mathsf{state}(r) =$ FAILED **or** no PENDING node has all-RESOLVED parents **then**
10: **return** v
11: **end if**
12: **end if**
13: $\mathcal{B} \leftarrow R$
14: **for each** $n \in \mathcal{B}$: $\mathsf{state}(n) \leftarrow$ EXECUTING
15: $\hat{v}_\mathcal{B} \leftarrow \mathsf{NodeProc}_\theta(\{(\tau_n, \mathsf{spec}_n, (v_p)_{p \in \mathsf{pa}_n}) : n \in \mathcal{B}\})$
16: **for** $n \in \mathcal{B}$ **do**
17: **if** $\hat{v}_{\mathcal{B},n} \neq \perp$ **then**
18: $v_n \leftarrow \hat{v}_{\mathcal{B},n}$; $\mathsf{state}(n) \leftarrow$ RESOLVED
19: **else**
20: $\mathsf{state}(n) \leftarrow$ FAILED
21: **end if**
22: **end for**
23: **end loop**

uniquely determined by G, σ, θ, and the inductive hypothesis. Determinism of $\mathsf{NodeProc}_\theta$ fixes v_n.

The batch composition $\mathcal{B}$ does not affect any individual node's input (Lemma 3.2), so the same value is produced regardless of how ready nodes are batched. □

Theorem 3.5 (Order independence)**.** *Under the hypothesis of Theorem 3.4, the value assignment produced by Algorithm 1 is invariant under any choice of $\mathcal{B} \subseteq R$ at each iteration, provided $\mathcal{B} \neq \emptyset$ whenever $R \neq \emptyset$.*

Proof. By Lemma 3.2, the input presented to $\mathsf{NodeProc}_\theta$ at any node n depends only on τ_n, spec_n, and parent values—not on other members of the dispatched batch. The set of nodes that ever reach RESOLVED is determined by the predicate of line 4 plus the absence-of-failure condition; under a deterministic processor that does not return $\perp$, every non-leaf node eventually has all parents resolved by induction on depth, and the output values match across schedules by Theorem 3.4. □

The architecture commits only to dependency-respecting evaluation. Among all schedules that respect dependencies, the executor's output is invariant.

Theorem 3.6 (Termination)**.** *Algorithm 1 terminates in at most $|V|$ iterations of the main loop, with the number of $\mathsf{NodeProc}_\theta$ invocations bounded by $|V| - |V_{\text{leaf}}|$.*

Proof. At each iteration with $R \neq \emptyset$, at least one node transitions from PENDING to READY at line 5, and subsequently to RESOLVED or FAILED. We claim $R \neq \emptyset$ whenever the loop has not yet returned. The set of PENDING nodes forms a sub-DAG of G; among the PENDING nodes,

those whose parents lie outside the PENDING set form a nonempty set provided the PENDING set is nonempty and the parent relation is acyclic. These nodes are precisely R. The loop therefore terminates after at most $|V|$ iterations, with each non-leaf node invoking the processor at most once. □

Theorem 3.7 (Soundness against the oracle)**.** *Let* $\mathsf{NodeProc}_\theta$ *be deterministic and primitive-correct: for every primitive* τ *and every typed input tuple* $(v_1, \ldots, v_{k_\tau})$*, the output of* $\mathsf{NodeProc}_\theta$ *on input* $(\tau, \cdot, (v_1, \ldots, v_{k_\tau}))$ *equals* $f_\tau(v_1, \ldots, v_{k_\tau})$*. Then for any reasoning task* (G, σ)*, the value* v_r *returned by Algorithm 1 equals the oracle ground-truth answer.*

Proof. Induction on $\mathsf{depth}(n)$. For $\mathsf{depth}(n) = 0$, $v_n = \sigma(n) = \hat{v}_n$ from line 2. For $\mathsf{depth}(n) > 0$, by the inductive hypothesis $v_p = \hat{v}_p$ for all $p \in \mathsf{pa}_n$. Primitive correctness of $\mathsf{NodeProc}_\theta$ yields $v_n = f_{\tau_n}(v_{p_1}, \ldots, v_{p_{k_{\tau_n}}}) = f_{\tau_n}(\hat{v}_{p_1}, \ldots, \hat{v}_{p_{k_{\tau_n}}}) = \hat{v}_n$. Applying to $n = r$ completes the proof. □

Theorem 3.7 isolates the only source of error in the executor: the only way the executor can deviate from oracle evaluation is if the processor is wrong on some primitive in the graph. This isolation is what makes the depth-decoupling analysis in Section 5 possible.

3.4 The node processor as a learning target

The node processor $\mathsf{NodeProc}_\theta$ is a small Transformer. Its input at a node n consists of a type embedding for τ_n, an encoding of spec_n, and the resolved values of pa_n formatted with type-aware delimiters. Its output is a value of type out_{τ_n} produced through a type-appropriate head: regression for continuous types, classification for finite types, copy mechanisms for variable-length structured types.

The processor has no access to a global graph embedding, no access to the depth or index of the node it is processing, and no access to a history vector summarising prior nodes. We refer to the tuple $(\tau_n, \mathsf{spec}_n, (v_p)_{p \in \mathsf{pa}_n})$ as the *local context* of n, and this is precisely what the processor sees. The consequences of this design for generalisation are developed in Section 6.

3.5 Iteration, recursion, and the layered treatment of control flow

The executor's view is acyclic by construction, but the system as a whole is not restricted to feed-forward computation. Iterative reasoning, recursion, and search-with-backtracking are expressible at one of three layers above the executor.

- *Iteration as graph depth.* A computation requiring k sequential applications of a primitive appears in the executor's view as a chain of k notion nodes, each invoking the shared processor once. This is the same unrolling that converts a recurrent neural network's source-level cycle into an acyclic computation graph at training time. The parameter count of the processor is unchanged; only the number of node invocations grows.

- *Iteration in the graph-construction policy.* When the construction of the DAG is itself learned (Section 7), the policy's control flow is unconstrained: the policy may loop over candidate expansions, condition on intermediate values, and decide adaptively whether to extend the current chain. The DAG it emits is acyclic; the procedure that emits it need not be.

- *Iteration in composite primitives.* A primitive's semantics is an arbitrary total function (Definition 2.2). Nothing requires a primitive to be implemented by a single forward pass of the neural processor. A primitive may be implemented symbolically, or as a procedural macro that expands at runtime into a sub-DAG of size dependent on its inputs (a FOLD, MAP, or WHILE combinator). Such composite primitives preserve the executor's acyclic invariant by producing acyclic sub-graphs on each invocation.

What the architecture excludes is *cyclic execution traces*—the executor's view itself containing a cycle. This exclusion is deliberate: cyclic traces require fixed-point semantics, lose deterministic termination, lose the order-independence of Theorem 3.5, and break the soundness argument of Theorem 3.7. The same exclusion is made (under different names) by every modern computation-graph framework. Loops live in source code or in the construction layer; the trace is always a DAG.

4 Architectural Comparison

The executor's inductive bias is best understood by contrast with two natural alternatives: a weight-tied graph transformer with a causal ancestor mask, and a sequence transformer over the serialised DAG. Both alternatives have access to identical input information; they differ in how they process it.

4.1 Weight-tied graph transformer with causal ancestor masking

Definition 4.1 (Weight-tied graph transformer)**.** Fix a topological ordering $\pi : V \to \{1, \ldots, N\}$ of G. The weight-tied graph transformer with causal ancestor masking computes

$$\begin{aligned} h_n^{(0)} &= \mathsf{Embed}(\tau_n, \mathsf{spec}_n) \quad \text{for each } n \in V, \\ h_n^{(t+1)} &= \mathsf{Layer}_\theta\Big(h_n^{(t)};\ \{h_{n'}^{(t)} : n' \in \mathsf{Ancestors}_\pi(n)\}\Big), \quad t = 0, \ldots, T-1, \\ v_n &= \mathsf{Head}_{\mathsf{out}_{\tau_n}}(h_n^{(T)}), \end{aligned}$$

where $\mathsf{Ancestors}_\pi(n) = \{n' : \pi(n') < \pi(n)\}$, Layer_θ is a self-attention block restricted by an ancestor mask with parameters shared across t, and Head_σ is a type-specific output head.

The defining features of this architecture are (i) per-layer parameters θ are shared across t, mirroring the executor's reuse of the processor; (ii) attention is restricted to topological ancestors, preserving the causal structure of the DAG; (iii) all node values are computed jointly across T rounds rather than node by node in dependency order.

Property (iii) is the substantive difference from the executor. A node's representation $h_n^{(t)}$ at layer $t > 0$ depends on layer-t representations of ancestors, not on the resolved values of parents. The architecture has T rounds in which to propagate information; nothing enforces that ancestors are fully resolved before descendants are updated.

4.2 Sequence transformer over the serialised DAG

Definition 4.2 (Sequence transformer over the serialised DAG)**.** Let $\mathsf{Serialize}(G)$ be a canonical token sequence encoding of G: for each node in topological order, emit tokens for its identifier, type, spec, and parent identifiers, separated by delimiters. The sequence transformer baseline passes $\mathsf{Serialize}(G)$ through a standard sequence transformer (decoder-only or encoder–decoder) and produces values via type-specific heads at the positions corresponding to each node.

The sequence transformer has the same information as the other two architectures but presented as a flat token sequence with no explicit graph structure beyond what positional embeddings and learned attention can recover.

4.3 Expressivity containment

The executor is a strictly more constrained function class than the weight-tied graph transformer.

Proposition 4.3 (Expressivity containment)**.** *For every choice of executor parameters θ_{GSRE} there exist weight-tied graph transformer parameters θ_{GT} and a depth $T \geq \mathsf{depth}(G)$ such that for every well-formed notion DAG G with $\mathsf{depth}(G) \leq T$, the value assignment produced by the executor equals the value assignment produced by the graph transformer.*

Proof. Set $T \geq \mathsf{depth}(G)$. Construct the layer so that at layer t, a node n with $\mathsf{depth}(n) = t$ attends to its parents (already updated to their final values at layer t by the inductive construction), copies their values into its working state, and applies $\mathsf{NodeProc}_{\theta_{\mathrm{GSRE}}}$ to compute its own value. For all $t' > t$, the node's representation passes through unchanged. The ancestor mask permits this attention pattern because parents are ancestors.

Conversely, the graph transformer has degrees of freedom the executor does not: nothing prevents a layer from blending information from non-parent ancestors, nor from updating a node based on layer-t values of nodes whose semantic values have not yet been computed. These functions are realisable by the graph transformer but not by the executor. □

The containment is what we are after. The executor's smaller hypothesis class encodes the prior that computation respects dependencies; the graph transformer must learn this prior from data.

5 Information-Theoretic Properties

We now characterise two properties of the executor: its per-step information budget is independent of graph depth, and under a stochastic processor with bounded per-invocation error, the executor's success probability decays geometrically in depth with no growth in parameters.

5.1 Per-step information budget

Let $\mathsf{Enc} : \mathcal{V} \to \{0,1\}^*$ be a fixed bit encoding of values, and write $|\mathsf{Enc}(v)|$ for the length of the encoding of v. The primitive description length of a node n is

$$\ell(n) \;=\; |\mathsf{Enc}(\tau_n)| + |\mathsf{Enc}(\mathsf{spec}_n)| + \sum_{p \in \mathsf{pa}_n} |\mathsf{Enc}(v_p)|.$$

Proposition 5.1 (Per-step bounded input)**.** *The input length presented to the processor at any node n is exactly $\ell(n)$, and*

$$\ell(n) \;\leq\; |\mathsf{Enc}(\tau_n)| + |\mathsf{Enc}(\mathsf{spec}_n)| + k_{\tau_n} \cdot L_{\max},$$

where $L_{\max} = \max_{v \in \mathcal{V}} |\mathsf{Enc}(v)|$. If $L_{\max}$ and $k_{\max} := \max_{\tau \in \mathcal{L}} k_\tau$ are finite, the per-step input length is $\mathcal{O}(k_{\max} L_{\max})$, independent of $|V|$ and $\mathsf{depth}(G)$.

Proof. Lemma 3.2 gives the exact input enumeration. Bounding each parent value by $L_{\max}$ yields the second inequality. □

The sequence transformer's per-step input is $\Theta(|V| \cdot L_{\max})$; the weight-tied graph transformer's per-layer attention scope is $\Omega(d)$ in a chain of depth d. The executor is the only one of the three architectures with a per-step input length independent of graph size.

5.2 Capacity–depth decoupling

The unconditional architectural property is the per-step bound of Proposition 5.1: at every node, the processor sees an input of size $\mathcal{O}(k_{\max} L_{\max})$, and the parameter count of $\mathsf{NodeProc}_\theta$ is fixed by the architecture and independent of $|V|$ and $\mathsf{depth}(G)$. Depth grows the number of processor invocations but not the per-invocation cost or capacity.

Translating this architectural property into an end-to-end accuracy bound requires a model of how per-primitive errors compose. We give first the cleanest such model and then state honestly the assumptions it makes.

Theorem 5.2 (Capacity–depth decoupling under independent primitive errors)**.** *Suppose* $\mathsf{NodeProc}_\theta$ *has per-invocation error bound* ε*:*

$$\sup_{\tau,(v_1,\ldots,v_{k_\tau})} \mathbb{P}\big[\mathsf{NodeProc}_\theta(\tau,\cdot,(v_1,\ldots,v_{k_\tau})) \neq f_\tau(v_1,\ldots,v_{k_\tau})\big] \;\leq\; \varepsilon,$$

and suppose further that error events at distinct invocations are independent. Then for any well-formed notion DAG $G = (V, r)$*,*

$$\mathbb{P}\big[v_r = \hat{v}_r\big] \;\geq\; (1-\varepsilon)^{|V|-|V_{\text{leaf}}|}.$$

For chain-shaped DAGs of depth d *with no shared subexpressions, this specialises to* $\mathbb{P}[v_r = \hat{v}_r] \geq (1-\varepsilon)^d$*.*

Proof. Conditional on every non-leaf node having received its correct parent values, each invocation independently succeeds with probability at least $1-\varepsilon$. The joint success of all $|V|-|V_{\text{leaf}}|$ non-leaf invocations therefore has probability at least $(1-\varepsilon)^{|V|-|V_{\text{leaf}}|}$, and this event implies $v_r = \hat{v}_r$. □

Remark 5.3 (Independence is the load-bearing assumption)*.* The independence assumption is unrealistic in practice. A shared-parameter processor has shared failure modes: if $\mathsf{NodeProc}_\theta$ is systematically wrong on inputs of a certain shape (e.g., near-zero arguments to a multiplicative primitive, edge cases of a string operation), every node whose local context falls in that region fails together. Theorem 5.2 is therefore best read as a baseline against which empirical decay is to be measured, not as a tight predictor of end-to-end accuracy.

The honest characterisation of the architectural payoff has two parts. (i) The unconditional part: the per-step input is bounded by max arity, the per-step parameter count is fixed, and the processor's hypothesis class is depth-invariant. These are properties of the architecture, not of any error model. (ii) The conditional part: under the geometric decay model, depth degrades success probability multiplicatively rather than catastrophically. The empirical question is whether realistic error correlation structure is closer to the independent model or to the worst case of fully-coupled failure.

Remark 5.4 (Empirical replacement of the geometric model)*.* Let $\hat{\varepsilon}(c)$ denote the empirical error rate of the trained processor on local contexts of a specified shape c, and let $\rho(c, c')$ denote the empirical covariance of error events between invocations with contexts c and c'. The accuracy of a depth-d chain is then bounded by a function of $\hat{\varepsilon}(\cdot)$ and $\rho(\cdot,\cdot)$ that reduces to $(1-\varepsilon)^d$ when $\rho \equiv 0$ and tracks the true behaviour otherwise. A complete empirical characterisation of the architecture therefore measures both the marginal error and the error correlation structure, and reports the depth-decay curve directly rather than via the geometric proxy.

The bound of Theorem 5.2 is loose where errors at downstream nodes can absorb upstream errors (e.g., ADD_MOD followed by a COMPARE that ignores the perturbed bit) and tight where each primitive's output is fully determined by its parent values and error events factor.

Remark 5.5 (Failure mode of the sequence baseline)*.* A sequence transformer trained on length-L_{train} serialised DAGs and tested on length-L_{test} DAGs faces (i) positional extrapolation [9], (ii) attention dispersion across longer sequences [4], and (iii) loss of independence between primitive evaluations, since all node values are produced by a single coupled latent computation. There is no analogue of Theorem 5.2 in this setting.

Remark 5.6 (Failure mode of the graph transformer baseline). The weight-tied graph transformer has T layers of shared parameters; depth-d graphs require $T \geq d$ for information to propagate from leaves to root. Even when T is sufficient, per-layer attention is over $\Omega(d)$ ancestors in a chain, which couples primitive evaluations through softmax temperature and attention dispersion. The independence factorisation of Theorem 5.2 does not hold.

6 Generalisation Properties

6.1 Local-context invariance

Definition 6.1 (Local context)**.** The local context of a node n is the tuple

$$\mathsf{Ctx}(n) \;=\; (\tau_n,\ \mathsf{spec}_n,\ (v_p)_{p\in\mathsf{pa}_n}).$$

Proposition 6.2 (Locality of the processor)**.** *For any two well-formed DAGs G, G' and any nodes $n \in V_G$, $n' \in V_{G'}$ with $\mathsf{Ctx}(n) = \mathsf{Ctx}(n')$, the value produced by the executor at n equals the value produced at n'.*

Proof. Immediate from Lemma 3.2 and determinism of $\mathsf{NodeProc}_\theta$. □

The executor sees only local contexts. Aggregated quantities of the graph—size, depth, ancestor topology, sibling identity—never reach the processor.

6.2 Marginal invariance and depth generalisation

Let $\mathcal{D}_d$ denote a distribution over notion DAGs of depth d paired with leaf assignments. Let $\mathcal{M}_d$ denote the marginal distribution of local contexts induced by drawing $G \sim \mathcal{D}_d$ and then drawing an internal node uniformly.

Theorem 6.3 (Depth-independent per-node error under marginal invariance)**.** *If $\mathcal{M}_{d_1} = \mathcal{M}_{d_2}$ then the expected per-node error rate of the executor's processor under $\mathcal{D}_{d_1}$ equals that under $\mathcal{D}_{d_2}$.*

Proof. The per-node error of the processor is a function $\phi : \mathsf{Ctx} \to [0, 1]$ of the local context alone. The expected per-node error under $\mathcal{D}_d$ equals $\int \phi \, d\mathcal{M}_d$. Equality of the marginals gives equality of the integrals. □

Combining the two:

Corollary 6.4 (Depth generalisation under marginal invariance)**.** *Let $\mathcal{D}_{d_{\text{train}}}$ and $\mathcal{D}_{d_{\text{test}}}$ have equal local-context marginals. Let ε be the per-node processor error rate under $\mathcal{D}_{d_{\text{train}}}$. Then the executor satisfies*

$$\mathrm{Acc}_{\mathcal{D}_{d_{\text{test}}}}(G_{SRE}) \;\geq\; \mathbb{E}_{G\sim\mathcal{D}_{d_{\text{test}}}}\Big[(1-\varepsilon)^{|V_G|-|V_{G,\text{leaf}}|}\Big].$$

The right-hand side depends on d_{test} only through the size distribution of the test graphs, and decays geometrically rather than catastrophically.

The marginal-invariance assumption is non-trivial. It is approximately satisfied when leaf values are sampled uniformly, primitive choices induce uniform output distributions, and the primitive library is closed under composition in a way that preserves the marginal distribution of internal-node parent values. Modular arithmetic over a prime modulus is a clean special case: for p prime, $(\mathbb{Z}/p\mathbb{Z})^\times$ is cyclic of order $p-1$, and the induced output distributions of $\mathsf{ADD\_MOD}, \mathsf{SUB\_MOD}, \mathsf{MUL\_MOD}$ on independent uniform inputs remain uniform on $\mathbb{Z}/p\mathbb{Z}$ (or on $(\mathbb{Z}/p\mathbb{Z})^\times$ for the multiplicative case, conditioned on nonzero inputs). In such settings $\mathcal{M}_d$ is approximately invariant in d, and Corollary 6.4 applies directly.

For tasks that do not admit a closed-form invariance argument, the assumption is empirically testable. Sample a corpus of DAGs at each depth of interest, evaluate the oracle on each, and collect the induced empirical distribution $\widehat{\mathcal{M}}_d$ over local contexts. A divergence between $\widehat{\mathcal{M}}_{d_1}$ and $\widehat{\mathcal{M}}_{d_2}$ (KL, total variation, or maximum mean discrepancy under a chosen kernel on the context space) quantifies how far the assumption is from holding for that task. Reporting this divergence alongside any depth-generalisation measurement turns Corollary 6.4 from an unverifiable conditional into a calibrated statement: where the empirical divergence is small, the accuracy bound applies; where it is large, the bound is to be weakened by an amount controlled by the divergence. We refer to this measurement as the *marginal-invariance diagnostic* and treat it as a required companion to any depth-generalisation claim.

6.3 Failure modes of the baselines under depth shift

For the weight-tied graph transformer, the number of ancestors of a node grows with depth. Attention aggregates over this set, and the marginal distribution of attention input sets under $\mathcal{D}_d$ shifts with d. There is no fixed-size per-step input, so the local-context invariance argument breaks down.

For the sequence transformer, the serialised DAG length grows with $|V|$, which grows with depth. Position embeddings, attention dispersion, and softmax temperature interact with sequence length, and the marginal distribution of inputs to the network shifts with depth.

The executor avoids both failure modes by construction: the per-step input is bounded, and only local contexts ever reach the processor.

7 Learned Graph Construction

The executor as developed so far assumes a pre-specified notion DAG. The architecture extends to the case where the DAG is constructed on the fly by a learned policy over a fixed primitive library. We formalise this construction as a Markov decision process with deterministic transitions.

7.1 The graph-construction MDP

Definition 7.1 (Graph-construction MDP)**.** The graph-construction MDP for a task specification q over library $\mathcal{L}$ is the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

State. An element of $\mathcal{S}$ is a pair (G, q) where G is a partial well-formed notion DAG over $\mathcal{L}$ together with a partial value assignment v. The initial state is $s_0 = (\emptyset, q)$.

Action. An action is a triple $a = (n_{\text{select}}, \tau, b)$:

- n_{select} is either a PENDING node in G with an unfilled slot, or a fresh node identifier.
- $\tau \in \mathcal{L}$ is a primitive whose output type matches the demanded type at n_{select}.
- b binds each input slot of τ either to an existing resolved value in G's value pool, or to a new sub-goal of the demanded type.

A distinguished action `FAIL` terminates the episode.

Transition. $P((G, q), a)$ deterministically produces G' by attaching τ at n_{select} with bindings b, then runs the traversal engine to resolve any newly-ready nodes.

Reward. $R(s, a, s') = +1$ if s' contains a fully resolved root whose value equals the task ground truth; otherwise $R = 0$.

Discount. $\gamma \in (0, 1)$.

A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ defines a distribution over actions. The Graph Policy Network GPN_ϕ parameterises such a policy.

7.2 Action space cardinality

Proposition 7.2 (Per-step action space)**.** *At a state with m unfilled slots across all* PENDING *nodes and v resolved values in the value pool,*

$$|\mathcal{A}(s)| \;\le\; (m+1)\cdot|\mathcal{L}|\cdot(v+|\mathcal{L}|)^{k_{\max}}+1.$$

Proof. $m+1$ choices of expansion target (existing slot or new root sub-goal); $|\mathcal{L}|$ choices of primitive; for each of up to $k_{\max}$ input slots, a binding to one of v resolved values or $|\mathcal{L}|$ new sub-goals; one extra action for `FAIL`. □

For a library of size $|\mathcal{L}| = K$, maximum arity $k_{\max} = 2$, and a value pool of modest size, the action space is polynomial in the partial-graph state and tractable for standard policy-gradient methods.

7.3 Deterministic transitions and search

The transition function of Definition 7.1 is deterministic conditional on the seed of $\mathsf{NodeProc}_\theta$. This permits Monte Carlo tree search over expansion sequences with the policy network as prior and a value head as leaf evaluator, in the style of AlphaZero [19]. Tree search interacts cleanly with the executor because every rollout is itself a deterministic partial execution of the underlying GSRE.

7.4 Training signal

The reward is sparse: $+1$ only when a fully-resolved root matches the ground truth. Two standard mechanisms apply.

Behaviour cloning bootstrap. When ground-truth expansion traces are available (from oracle DAG generation at low depth), the policy can be initialised by supervised loss

$$\mathcal{L}_{\mathrm{BC}}(\pi_\phi) \;=\; -\mathbb{E}_{(s,a^*)}\big[\log \pi_\phi(a^* \mid s)\big],$$

where a^* is the oracle expansion action at s.

Policy-gradient refinement. Subsequent training proceeds via PPO [20] or analogous policy-gradient updates on the sparse-reward signal, possibly augmented with MCTS-improved policy targets [19]. Because transitions are deterministic, value estimates depend only on the policy itself and the (fixed-seed) executor.

7.5 Sample complexity of policy learning

The graph-construction MDP is structurally close to the AlphaZero-style setting of deterministic-transition single-agent search with sparse terminal reward and a small per-step action space, for which the sample-complexity behaviour of search-augmented policy iteration is empirically well-characterised. Two features make the present setting more favourable than the chess/Go regime in which those bounds were established: transitions are not merely deterministic but fully simulable by the executor itself, and the reward is verifiable by re-execution rather than by a learned value head. Two features make it harder: episodes can be longer because reasoning depth is unbounded, and the policy's value head must learn to predict success at intermediate partial graphs whose value is the joint product of all remaining primitive evaluations.

We do not give an end-to-end sample-complexity bound. Quantitative claims about the difficulty of training the graph policy are intentionally deferred to the empirical programme of Section 10, where they appear as a measurable diagnostic (learning curves with and without MCTS augmentation, parameterised by graph depth) rather than as a theorem.

8 Library Extension

The most ambitious extension of the architecture is the extension of the primitive library itself: identifying reusable subgraphs in successful traces, abstracting them into new primitives, and incorporating these into the library. The framework admits a clean formal statement of the problem.

8.1 Compositional compression

Let $\mathcal{T} = \{G_1, G_2, \dots\}$ be a collection of successfully-resolved notion DAGs. A subgraph abstraction is a typed parameterised graph template T together with embeddings $\iota_i : T \hookrightarrow G_i$ for each G_i in some $\mathcal{T}_T \subseteq \mathcal{T}$, where each ι_i preserves the structure of T up to substitution of free type variables and leaves.

Definition 8.1 (Compositional compression value)**.** The compression value of an abstraction T over $\mathcal{T}$ is

$$\Delta(T) \;=\; \sum_{i\,:\,T \hookrightarrow G_i} (\mathsf{count}_T(G_i) - 1) \cdot |T| \;-\; |T|,$$

where $\mathsf{count}_T(G_i)$ is the number of structurally-distinct embeddings of T in G_i, and $|T|$ is the description length of T.

The library-learning problem is to identify T with large $\Delta(T)$. It generalises subgraph-mining problems studied in the graph-data-mining literature [24] and connects directly to library learning in inductive program synthesis [16]. The novel features in the present setting are the typed parameterisation of abstractions and the requirement that abstractions remain executable by the shared processor.

8.2 Continual learning of the processor

Adding a new primitive τ_{K+1} to the library requires the processor to learn the new operation. Two constraints conflict:

- *Stability.* Per-primitive accuracy on $\{\tau_1, \dots, \tau_K\}$ must not degrade after training on τ_{K+1}.
- *Plasticity.* Time to reach $1 - \varepsilon$ accuracy on τ_{K+1} should be sublinear in K.

Standard tools—adapter layers [21], low-rank adaptation [22], elastic-weight consolidation [23], mixture-of-experts—trade these properties against each other. The shared-processor architecture commits to a single set of parameters for all primitives; reconciling stability and plasticity in this constrained setting is open.

Procedural primitives as a structural mitigation. A pragmatic alternative to demanding that the neural processor learn every new primitive is to admit *procedural primitives* into the library: primitives whose semantic function is implemented not by a forward pass of $\mathsf{NodeProc}_\theta$ but by a program over already-learned atomic primitives. A procedural primitive τ carries an expansion rule

$$\mathsf{Expand}_\tau \;:\; \mathcal{V}_{\mathsf{in}_\tau} \;\to\; (\text{sub-DAG over } \mathcal{L}),$$

which the executor invokes in place of $\mathsf{NodeProc}_\theta$ at any node of type τ. Procedural primitives may be parameterised by structural arguments (e.g. list length), may contain loops and conditionals in their expansion rule, and may be composed; what they may not do is introduce cycles in the resulting sub-DAG, so the executor's invariants are preserved. The stability–plasticity tradeoff in $\mathsf{NodeProc}_\theta$ is sidestepped entirely: the neural processor's parameter set is fixed, and library growth happens at a layer above it. This is the same mechanism by which programming-language standard libraries are extended without modifying the underlying interpreter, and it

connects directly to the wake–sleep library learning regime of [16] interpreted as procedural macros rather than learned atomic operations.

8.3 Verification of invented primitives

A new primitive τ_{K+1} induced by abstraction must be verified to compute a useful function. Two interpretations of usefulness are operationally distinct.

- *Compression-based.* τ_{K+1} is admitted to the library iff it reduces the description length of solutions on a held-out problem set by an amount exceeding the cost of its parameters.
- *Semantic.* τ_{K+1} must compute a well-defined function. Absent an external oracle, this function is whatever the processor produces, and correctness is a property of the abstraction process itself.

Compression-based verification is realisable and is the criterion used by DreamCoder. Semantic verification raises the foundational question of whether the system can construct test cases for its own invented primitives, which in the present setting reduces to whether the example traces from which τ_{K+1} was mined constitute a sufficient defining set.

8.4 Search versus abstraction

A library-extending agent must decide, for any novel problem, whether to search the existing library or to introduce a new primitive. The two options have distinct costs: search scales with the size of the action space at the relevant depth; abstraction scales with the size of the trace corpus.

9 Connections to Prior Work

The architecture connects to several established lines of work.

Compositional generalisation. The SCAN benchmark [1] and its successors [2, 3] established the empirical phenomenon that motivates this architecture. Subsequent work demonstrated architectural and data-side mitigations [10, 11, 12]. The present architecture mitigates the gap by structural enforcement.

Chain-of-thought and scratchpad reasoning. Wei et al. [6] and Nye et al. [7] expose intermediate reasoning steps as targets in token form. The notion DAG is the structural counterpart: the same intermediate steps, represented explicitly rather than implicitly in a token sequence.

Universal and recurrent transformers. Weight-tied transformers [8] are the sequence analogue of the weight-tied graph transformer of Section 4.1. Length-generalisation behaviour of recurrent architectures has been investigated by [5].

Neural module networks. Andreas et al. [13] construct computation graphs from a fixed module library by parsing the input question. The notion DAG generalises this to an arbitrary typed DAG with a shared module.

Program synthesis and library learning. The learned-graph-construction MDP of Section 7 is a neural program synthesis problem with a known DSL [14, 15]. The library-extension setting of Section 8 is most closely related to DreamCoder [16].

Graph neural networks. The weight-tied graph transformer is structurally close to message-passing GNNs [17, 18], differing in that GNN message passing updates all nodes at every layer regardless of whether their inputs have stabilised.

10 Empirical Programme

The theorems and bounds developed in the preceding sections are predicated on assumptions whose realism is itself an empirical question. We list here the minimal set of measurements that would tighten the document from a conditional statement of properties to a calibrated description of system behaviour. Each item maps onto a specific theorem and produces a diagnostic that either supports the theorem's antecedent or quantifies the gap.

1. *Per-primitive accuracy and error correlation structure.* Train $\mathsf{NodeProc}_\theta$ on the primitive library and measure (i) the marginal error rate $\hat{\varepsilon}(\tau, c)$ for each primitive τ and local-context shape c; (ii) the empirical covariance ρ of error events across invocations sharing a context shape; (iii) the realised depth-decay curve on chained primitives, to be compared against the $(1-\varepsilon)^d$ baseline of Theorem 5.2. This tests whether the architectural decoupling argument survives the realities of a shared-parameter neural network (Remark 5.3).

2. *Marginal-invariance diagnostic.* For each task family of interest, sample DAGs at depths $d \in \{1, \ldots, d_{\max}\}$, evaluate the oracle to populate parent values, and compute a divergence (KL or MMD) between the empirical context marginals $\widehat{\mathcal{M}}_d$. Report the divergence alongside any depth-generalisation accuracy curve. This calibrates Corollary 6.4 for the specific task: where the divergence is small, the bound applies; where it is large, the bound is to be weakened in proportion.

3. *OOD depth-generalisation curves.* Train the executor on shallow graphs ($d \leq 3$) and measure exact-match accuracy on test graphs of depth $d \in \{4, 5, 7, 9, 10\}$ across multiple seeds, comparing against the weight-tied graph transformer of Definition 4.1 and the sequence transformer of Definition 4.2 under matched training compute, parameter count, and primitive coverage. The OOD slope difference is the direct test of the architectural claim that explicit dependency scheduling improves depth generalisation beyond implicit causal masking.

4. *Graph-policy learning curves.* Train the graph-policy network of Section 7 under three regimes: pure policy gradient with sparse reward, policy gradient with behaviour-cloning warm-start, and AlphaZero-style MCTS-augmented policy iteration. Report sample efficiency and asymptotic success rate as a function of maximum task depth. This characterises the policy-learning difficulty acknowledged in Section 7.5.

Together these measurements close the loop between architecture and behaviour: items 1 and 2 calibrate the executor's idealised guarantees, item 3 tests the architectural prediction against alternatives, and item 4 measures the trainability of the construction layer above it. Any negative result in this programme localises the architectural assumption that fails, which is what makes the assumptions worth stating precisely.

11 Discussion

The executor's key inductive bias is the hard enforcement of dependency order. Among architectures with access to identical information, only the executor commits to the constraint that a node's value is computed once, after all of its inputs are available, by a function whose input is restricted to the node's local context. Weight-tied graph transformers approximate this constraint via causal ancestor masking; sequence transformers do not approximate it at all.

The consequence is a clean separation of concerns. The neural component is responsible only for executing primitives; the symbolic component is responsible for tracking dependencies and scheduling. Each can be analysed independently. The neural component's error is bounded by primitive-level training; the symbolic component's correctness follows by structural induction on the DAG.

The capacity–depth decoupling of Theorem 5.2 is the architectural payoff. A small processor trained to high primitive accuracy can execute arbitrarily deep graphs, with success probability decaying geometrically rather than catastrophically and with no growth in parameters. The local-context invariance of Proposition 6.2 ensures that this decoupling extends to graphs of depths not seen in training, provided the marginal distribution of local contexts is preserved.

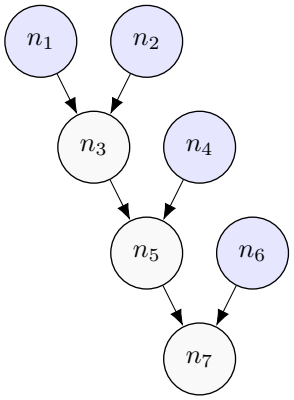
The extensions to learned graph construction and library extension preserve this structure. The construction policy is a separate neural component that decides what graph to build; the executor as developed here resolves whatever graph it is given. The library-extension problem becomes a problem of mining and verifying new abstractions, with the executor's structural guarantees serving as ground for the verification step.

A Notation Summary

Symbol	Meaning
$\mathcal{T}$	Set of types in the type system.
$\mathcal{V}_\tau$	Value domain of type τ.
$\mathcal{L}$	Primitive library. $\lvert\mathcal{L}\rvert = K$.
τ	A primitive; in_τ, out_τ, f_τ are its input/output types and semantics.
n	A notion: $(\mathsf{id}_n, \tau_n, \mathsf{spec}_n, \mathsf{pa}_n)$.
$G = (V, r)$	A notion DAG with vertex set V and root r.
$\mathsf{depth}(n)$	Compositional depth of n.
$\mathsf{depth}(G)$	Depth of the graph; $= \mathsf{depth}(r)$.
$\mathsf{state}(n)$	Execution state in $\{\textsc{Pending}, \textsc{Ready}, \textsc{Executing}, \textsc{Resolved}, \textsc{Failed}\}$.
v_n	Resolved value at n.
$\hat{v}_n$	Oracle ground-truth value at n.
$\mathsf{NodeProc}_\theta$	Shared neural node processor with parameters θ.
ε	Per-primitive error rate of $\mathsf{NodeProc}_\theta$.
k_τ	Arity of primitive τ. $k_{\max} = \max_\tau k_\tau$.
$\mathsf{Ctx}(n)$	Local context of n: $(\tau_n, \mathsf{spec}_n, (v_p)_{p \in \mathsf{pa}_n})$.
$\mathcal{D}_d$	Distribution over depth-d notion DAGs.
$\mathcal{M}_d$	Marginal distribution over local contexts under $\mathcal{D}_d$.
$\textsc{Gpn}_\phi$	Graph Policy Network with parameters ϕ.

B Worked Trace

We trace the executor on the DAG corresponding to $((3+5)-2)\cdot 7 \mod 101$ over a primitive library containing $\mathsf{CONST}_{\mathsf{Nat}}, \mathsf{ADD\_MOD}, \mathsf{SUB\_MOD}, \mathsf{MUL\_MOD}$ with the obvious semantics in $\mathbb{Z}/101\mathbb{Z}$. The DAG is shown in Figure 2.



$$n_1 = \mathsf{CONST}(3), \quad n_2 = \mathsf{CONST}(5),$$
$$n_3 = \mathsf{ADD\_MOD}(n_1, n_2), \quad n_4 = \mathsf{CONST}(2),$$
$$n_5 = \mathsf{SUB\_MOD}(n_3, n_4), \quad n_6 = \mathsf{CONST}(7),$$
$$n_7 = \mathsf{MUL\_MOD}(n_5, n_6).$$

Figure 2: The notion DAG for $((3+5)-2)\cdot 7 \mod 101$. The root is n_7; leaves are shaded, internal nodes unshaded.

Trace:

1. Initialisation. All leaves (n_1, n_2, n_4, n_6) are assigned constants and marked RESOLVED with values $3, 5, 2, 7$. All other nodes are PENDING.
2. Iteration 1. $R = \{n_3\}$. The processor receives input $(\mathsf{ADD\_MOD}, \cdot, (3, 5))$ and returns 8. $v_{n_3} = 8$, $\mathsf{state}(n_3) = \text{RESOLVED}$.
3. Iteration 2. $R = \{n_5\}$. The processor receives input $(\mathsf{SUB\_MOD}, \cdot, (8, 2))$ and returns 6. $v_{n_5} = 6$, $\mathsf{state}(n_5) = \text{RESOLVED}$.
4. Iteration 3. $R = \{n_7\}$. The processor receives input $(\mathsf{MUL\_MOD}, \cdot, (6, 7))$ and returns 42. $v_{n_7} = 42$, $\mathsf{state}(n_7) = \text{RESOLVED}$.
5. Termination. The root is RESOLVED; the loop exits and returns v with $v_r = 42$.

The processor was invoked three times. At each invocation its input was bounded by the description length of the primitive plus two parent values; no part of the input depended on the depth of the graph or the identity of nodes outside the direct parents.

References

[1] B. M. Lake and M. Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. *ICML*, 2018.

[2] N. Kim and T. Linzen. COGS: A compositional generalization challenge based on semantic interpretation. *EMNLP*, 2020.

[3] D. Hupkes, V. Dankers, M. Mul, and E. Bruni. Compositionality decomposed: How do neural networks generalise? *JAIR*, 67, 2020.

[4] C. Anil et al. Exploring length generalization in large language models. *NeurIPS*, 2022.

[5] A. Schwarzschild et al. Can you learn an algorithm? Generalizing from easy to hard problems with recurrent networks. *NeurIPS*, 2021.

[6] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

[7] M. Nye et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv:2112.00114*, 2021.

[8] M. Dehghani et al. Universal transformers. *ICLR*, 2019.

[9] O. Press, N. A. Smith, and M. Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *ICLR*, 2022.

[10] J. Russin et al. Compositional generalization in a deep seq2seq model by separating syntax and semantics. *arXiv:1904.09708*, 2019.

[11] R. Csordás, K. Irie, and J. Schmidhuber. The devil is in the detail: Simple tricks improve systematic generalization of transformers. *EMNLP*, 2021.

[12] J. Andreas. Good-enough compositional data augmentation. *ACL*, 2020.

[13] J. Andreas, M. Rohrbach, T. Darrell, and D. Klein. Neural module networks. *CVPR*, 2016.

[14] S. Gulwani, O. Polozov, and R. Singh. Program synthesis. *Foundations and Trends in Programming Languages*, 4(1–2), 2017.

[15] J. Devlin et al. RobustFill: Neural program learning under noisy I/O. *ICML*, 2017.

[16] K. Ellis et al. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI*, 2021.

[17] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *ICLR*, 2017.

[18] P. Veličković et al. Graph attention networks. *ICLR*, 2018.

[19] D. Silver et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 2018.

[20] J. Schulman et al. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.

[21] N. Houlsby et al. Parameter-efficient transfer learning for NLP. *ICML*, 2019.

[22] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.

[23] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *PNAS*, 114(13), 2017.

[24] T. Washio and H. Motoda. State of the art of graph-based data mining. *ACM SIGKDD Explorations*, 5(1), 2003.